

MACHINE LEARNING PROJECT EXPLANATION

Startup Profit Prediction Using SVR

A practical guide to the data preparation, Support Vector Regression model, evaluation process, benefits, and production-readiness fixes.

Based on the notebook **SVM_1790175737086.ipynb**

Dataset: **50_Startups_dataset.csv**

1. Project Summary

This project predicts a startup's expected profit using its R&D spending, administration costs, marketing spending, and state. The notebook uses **Support Vector Regression (SVR)**, which is the regression form of Support Vector Machines.

Important distinction: the notebook is named “SVM”, but it predicts a continuous value called `Profit`. Therefore it uses `SVR`, not a classification model such as `SVC`.

Inputs

- R&D Spend
- Administration
- Marketing Spend
- State

Prediction target

Profit

A continuous numeric value, making this a regression problem.

End-to-end workflow

1. Load the startup dataset with pandas.
2. Encode the categorical state column as numeric features.
3. Separate input features from the profit target.
4. Split the data into training and testing sets.
5. Standardize the features so each has a comparable scale.
6. Train a linear-kernel SVR model.
7. Generate predictions for unseen test data.
8. Measure performance using the R^2 score.
9. Save the complete model pipeline for future use.

2. Code Explanation

Loading and preparing the data

```
import pandas as pd

dataset = pd.read_csv("50_Startups_dataset.csv")
dataset = pd.get_dummies(dataset, drop_first=True)
```

`read_csv()` loads the data into a DataFrame. The `State` column contains text categories, so `get_dummies()` converts them into numeric indicator columns. Using `drop_first=True` keeps one state as the reference category and avoids redundant columns.

Separating features and target

```
independent = dataset[
    ["R&D Spend", "Administration", "Marketing Spend",
     "State_Florida", "State_New York"]
]
dependent = dataset["Profit"]
```

The independent variables are the model inputs. The dependent variable is the value the model must learn to predict. Keeping `dependent` as a one-dimensional Series avoids the `DataConversionWarning` produced by a column-shaped target.

Splitting and scaling

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

x_train, x_test, y_train, y_test = train_test_split(
    independent,
    dependent,
    test_size=0.2,
    random_state=1
)

scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)
```

The test set is kept separate so it represents unseen data. The scaler learns its mean and standard deviation only from the training set. Applying the same transformation to the test set prevents data leakage.

Training and evaluating SVR

```
from sklearn.svm import SVR
from sklearn.metrics import r2_score

regressor = SVR(kernel="linear", C=10000)
regressor.fit(x_train, y_train)

y_pred = regressor.predict(x_test)
```

```
score = r2_score(y_test, y_pred)
print(f"R2 score: {score * 100:.2f}%")
```

The linear kernel looks for a linear relationship in the scaled feature space. The `C` parameter controls how strongly the model penalizes errors. A higher value prioritizes fitting the training data more closely.

3. Model Benefits

SVR is useful when the target is numeric and the dataset is small or medium-sized. Its main strengths in this project are:

Benefit	Why it matters here
Works for continuous predictions	Profit is a numeric amount rather than a class label, so SVR directly matches the problem.
Epsilon-insensitive loss	Small prediction errors inside the epsilon margin are not treated as important, which can help the model focus on meaningful deviations.
Regularization	The <code>C</code> parameter provides control over the trade-off between fitting the training data and keeping the model generalizable.
Effective on small datasets	The 50 Startups dataset is small, a setting where SVR can be practical and accurate without requiring a large neural network.
Kernel flexibility	The current model uses a linear kernel, but other kernels such as RBF can model more complex relationships when justified by validation.
Feature-scale awareness	Standardization makes spending columns with different magnitudes comparable and is essential for distance and margin-based models.

Trade-offs and limitations

- SVR can become slow on very large datasets.
- The linear kernel cannot capture strongly nonlinear patterns by itself.
- The reported score comes from one train-test split; cross-validation is better for a dependable estimate.
- SVR is less interpretable than a simple linear regression model.
- The value of `C` should be selected using validation rather than assumed to be optimal.

Notebook result

95.89%

R^2 score reported by the notebook on its test split.

Interpret carefully

This is an encouraging result, but it is not a guarantee of future performance. Compare cross-validation scores and error metrics before using the model for decisions.

4. Reliable, Corrected Implementation

The safest design is to keep preprocessing and the regressor in one pipeline. That way, new startup data receives exactly the same transformations used during training.

```
import joblib
import pandas as pd

from sklearn.compose import ColumnTransformer
from sklearn.metrics import mean_absolute_error, r2_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.svm import SVR

dataset = pd.read_csv("50_Startups_dataset.csv")

X = dataset.drop(columns=["Profit"])
y = dataset["Profit"]

numeric_features = [
    "R&D Spend",
    "Administration",
    "Marketing Spend"
]
categorical_features = ["State"]

preprocessor = ColumnTransformer(
    transformers=[
        ("numeric", StandardScaler(), numeric_features),
        (
            "categorical",
            OneHotEncoder(drop="first", handle_unknown="ignore"),
            categorical_features
        )
    ]
)

model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("regressor", SVR(kernel="linear", C=10000))
    ]
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1
)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("R² score:", r2_score(y_test, y_pred))
print("Mean absolute error:", mean_absolute_error(y_test, y_pred))

joblib.dump(model, "startup_profit_svr.joblib")

new_startup = pd.DataFrame([
    "R&D Spend": 5000,
    "Administration": 6666,
    "Marketing Spend": 8888,
    "State": "New York"
])
```

```
predicted_profit = model.predict(new_startup)[0]  
print(f"Predicted profit: {predicted_profit:.2f}")
```

5. Fixes Before Publishing

Issue	Correction
Column-shaped target	Use <code>dataset["Profit"]</code> rather than <code>dataset[["Profit"]]</code> .
Inconsistent model filename	The notebook saves <code>PaL_model.sav</code> but later loads <code>P&L_model.sav</code> . Use one filename consistently.
Scaler is not saved with the model	Save a complete scikit-learn Pipeline ; otherwise raw future inputs will not be transformed correctly.
Undefined variable	The interactive cell checks <code>predicted_input</code> even though it creates <code>prediction_input</code> .
Undefined predicted value	Assign <code>predicted_profit = model.predict(prediction_input)[0]</code> before testing whether it is positive or negative.

Recommended next steps

- Use cross-validation to estimate generalization performance.
- Report both R^2 and mean absolute error.
- Compare the linear SVR with linear regression, random forest regression, and RBF-kernel SVR.
- Tune `C` and `epsilon` using `GridSearchCV`.
- Keep the dataset, notebook, saved pipeline, and README together in the GitHub repository.

Conclusion: this project demonstrates a complete regression workflow—from categorical encoding and feature scaling to model training, evaluation, persistence, and future prediction. The strongest production improvement is saving preprocessing and SVR as one pipeline.

Prepared from the supplied startup-profit SVR notebook. The performance figure is the notebook's reported result and should be rechecked after applying the corrected pipeline.